

HW3: Simple MPI Circle Renderer - Report

Team 23

R14922159 Lin, Hsin-Yu R14922112 Chen, Li-Jung R14945029 Lin, Wan-Hsuan

Describe the input sizes and how they affect runtime (image size, number of circles).

We will discuss the relation between input size and runtime by serial and MPI.

Serial

As shown in Figure 1, runtime is positively correlated with the number of circles but unrelated to the image size.

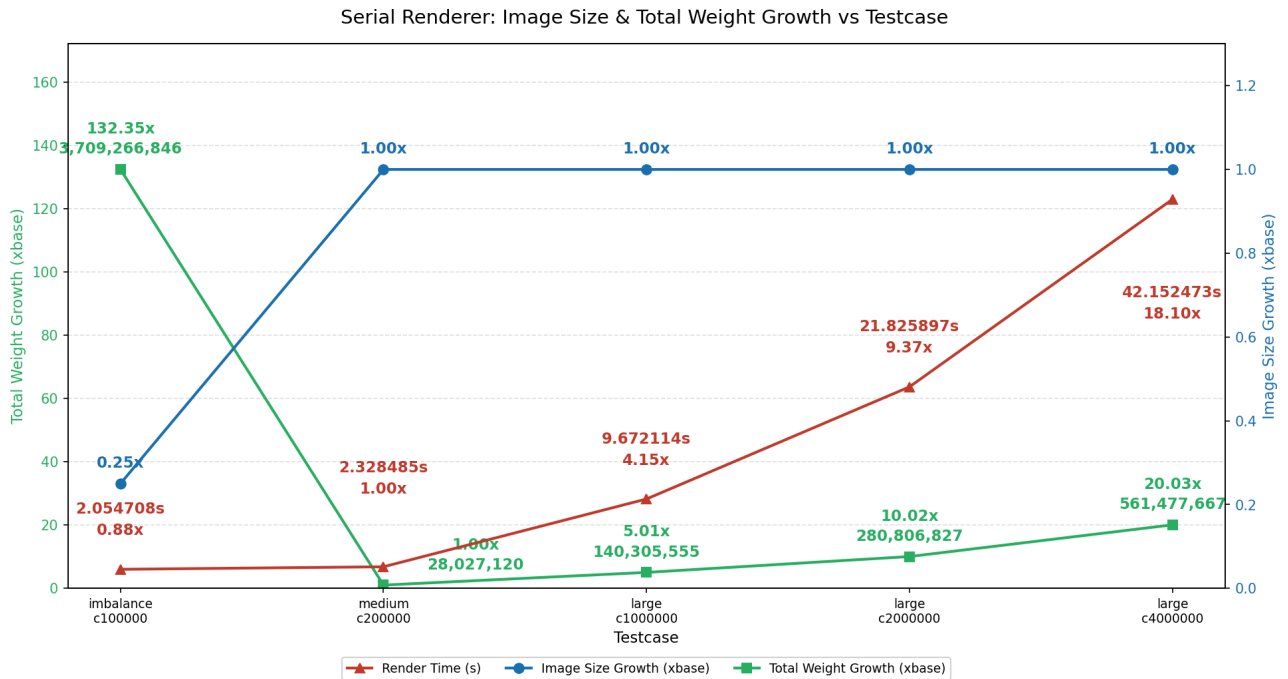


Figure 1: Execution time vs. image size & total weight with serial renderer, `medium_c2000000` as base.

Image size is only related to the write operation after calculating the RGB values of all pixels, but in our test cases, the image size is negligible compared to the computational cost of pixel RGB values.

The total weight and number of circles in the last four test cases show almost linear growth, suggesting that the circle size distribution in these test cases are relatively similar. `imbalance_c100000` has an extremely high total weight, but its execution time does not increase significantly, indicating that it has many large circles, and a large portion of these large circles fall outside the image, thus preventing a dramatic increase in computational cost.

MPI

As shown in Figure 2, overall runtime is positively correlated with the number of circles and unrelated to image size.

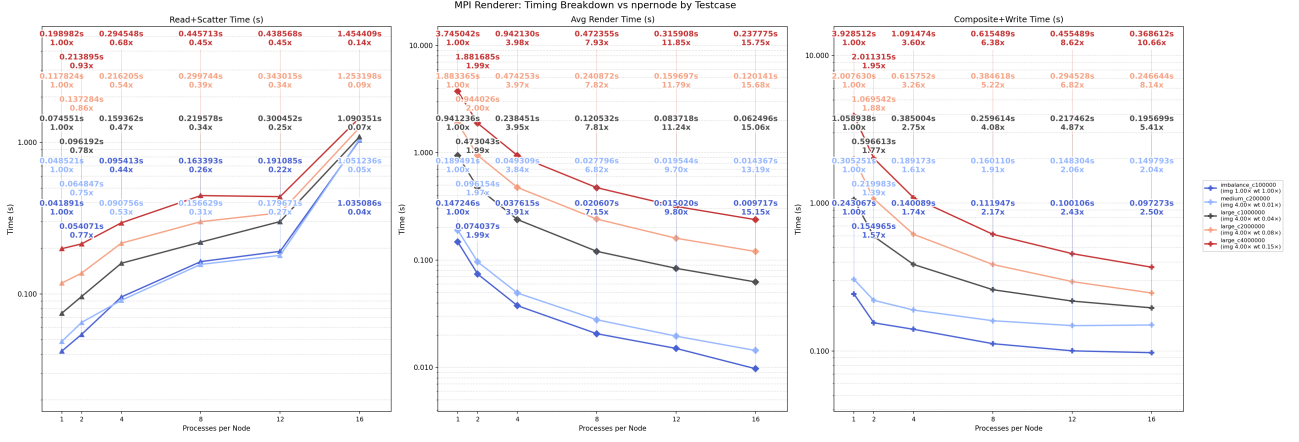


Figure 2: npnnode vs. Read+Scatter Time & Avg. Render Time & Composite+Write Time

- The “Read+Scatter time” is positively correlated with npnnode. Remarkably, the communicational overhead during scattering starts to scale up when npnnode is greater than 14, as shown in Figure 3.

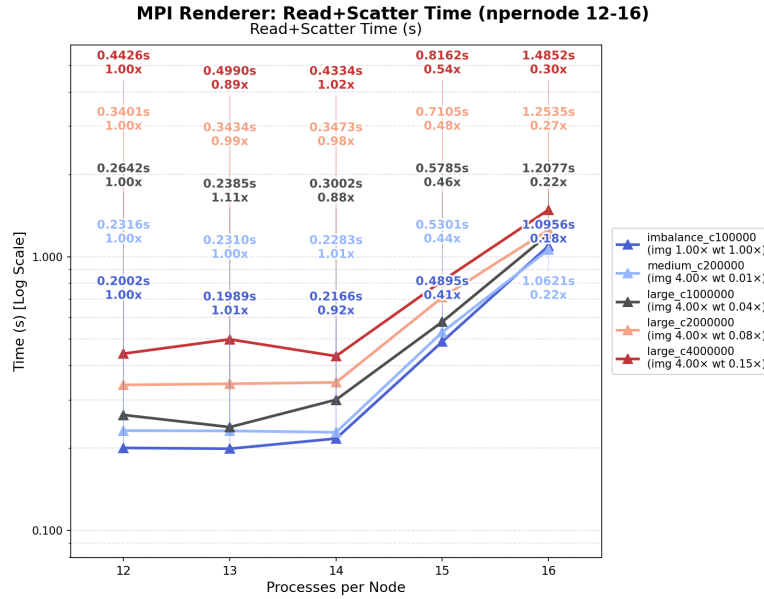


Figure 3: npnnode 12-16 vs. Read+Scatter Time

- The “Per-rank render time” is negatively correlated with npnnode, as the per-rank amount of computation decreases. The speedup among the test cases when npnnode=16 ranges from 13.19 to 15.68, revealing that a linear correlation (good scalability) can be seen in this task.
- The “Composite + write time” is negatively correlated with npnnode. For some test cases with smaller problem size, the speedup increases at a decreasing rate as npnnode increases. For deeper analysis, writing image requires a fixed amount of time while the render time is almost linearly scalable; therefore, we can deduce that `MPI_Reduce()` will slightly increase as npnnode increases, meaning that it is not completely parallelizable.

Report runtime vs. number of processes (strong scaling).

For the given testcases, the overall runtime generally decreases as the number of processes increases as shown in Figure 4. However, with the number of processes in range of 8 to 16, a decreasing trend in speedup is shown, suggesting that the computational amount of each process is too low to offset the overhead of “read+scatter+composite+write”. In a nutshell, there’s a sweet spot for each test case, where the communication overhead has not yet exceeded the benefit of parallelization.

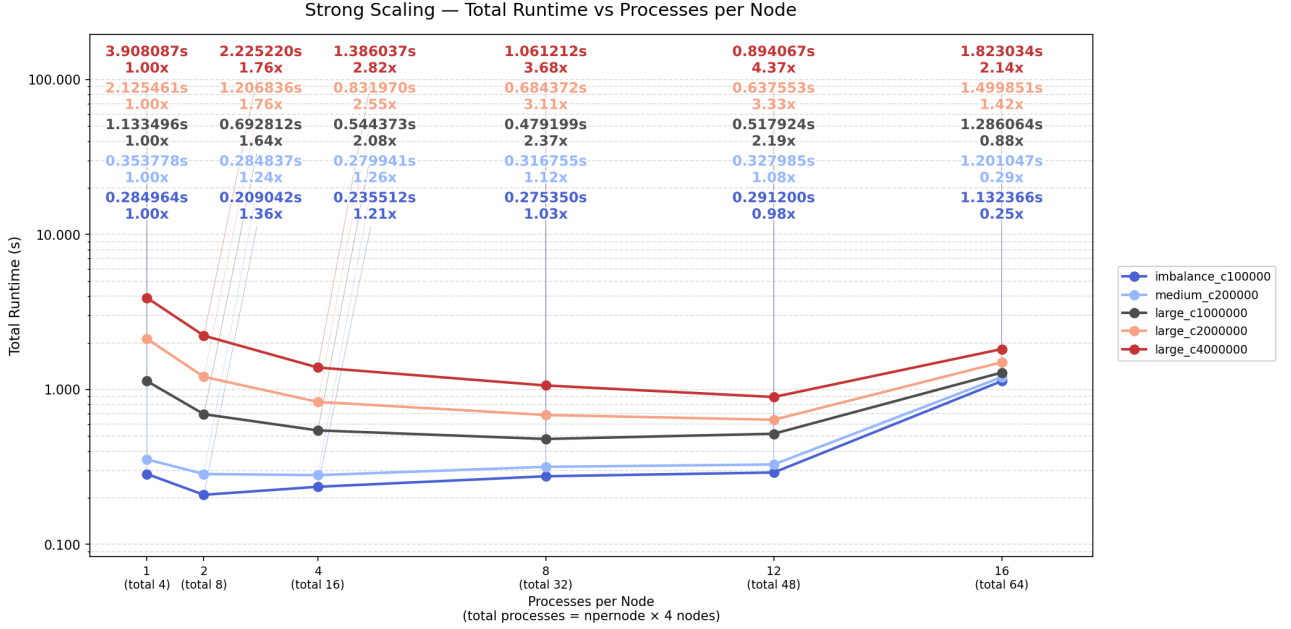


Figure 4: npnode vs. Total Runtime

Report runtime vs. problem size per process (weak scaling).

As shown in Figure 5, we will illustrate the weak scalability tests by `large_c1000000`, `large_c2000000`, and `large_c4000000`, with 1:2:4 npnode configuration.

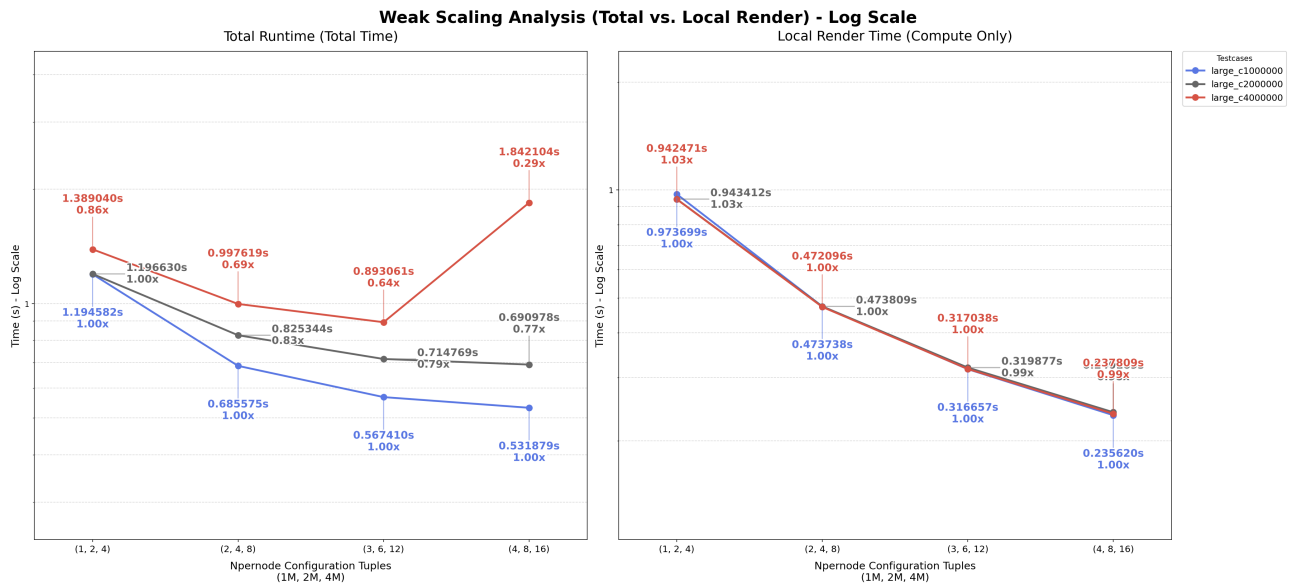


Figure 5: npnode vs. Total Runtime & Local Render Time

The total time decreases as npnode increases, but larger test cases still take slightly longer time,

presumably due to the overhead of `MPI_Scatterv()`.

The average local render time decreases as `npernode` increases, and the time for each test case is almost the same, further indicating that the increased total time is indeed due to the overhead of read+scatter time. In conclusion, the per-rank computation is almost totally parallelizable, displaying a good weak scalability.

Report speedup and efficiency relative to the serial baseline.

The speedup and efficiency are shown in Figure 6.

- **Speedup:** As the total number of processes increases, speedup initially increases and then decreases. Test cases with larger amounts of computation have higher maximum speedup values and slower speedup declines; conversely, test cases with smaller amounts of computation have lower maximum speedup values and start declining earlier. This indicates that more processes can effectively distribute computational load, but the non-parallelizable portion remains the bottleneck for speedup.
- **Efficiency:** Efficiency decreases significantly with increasing total processes, indicating that the impact of the non-parallelizable portion is substantial.

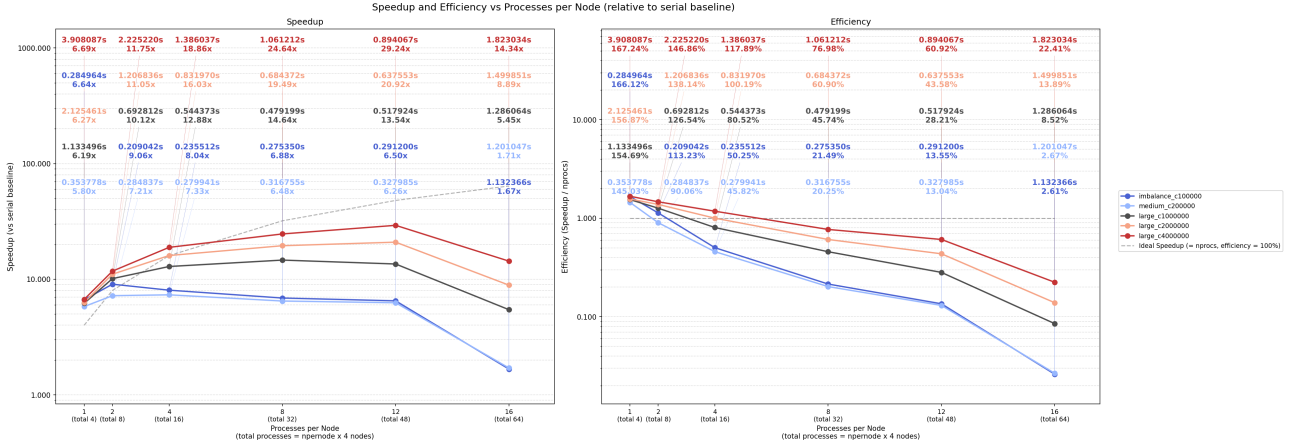


Figure 6: npernode vs. Speedup & Efficiency

What optimization techniques you applied?

- As shown in Figure 7, we packed the RGB value and count index of each row into an unsigned 64-bit integer, where each RGB value was stored in an unsigned 8-bit integer and the count index was stored in an unsigned 32-bit integer. By packing the count with the first 32 bits, we can use `MPI_Reduce()` to get the maximum value of each pixel; therefore, we know the last value assigned to that pixel, omitting the need of sequential `MPI_Recv()` of every rank except rank 0.

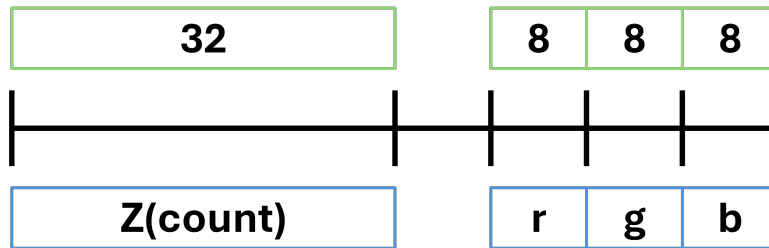


Figure 7: Count index, and RGB values packed in 64 bits.

- The `imbalance_c1000000` test case has a small number of circles with imbalance size distribution. As the number of npernode increases and the target weight required by each process decreases, load imbalance is more likely to occur. Other test cases have a larger number of circles with more uniform size distribution, resulting in higher overall load balance efficiency, but there is a tendency for it to decrease as the number of npernode increases. The `large_c4000000` and `large_c2000000` test cases show that the more amount of data and the more uniform the distribution, the less likely the load balance efficiency will decrease due to an increase in npernode.

Anything you want to share.

Non-parallelizable

In order to write the image correctly, we need to find the count index of the last assigned circle of each pixel. This part of the task is non-parallelizable and cannot be avoided.

Load Balance

$$\text{Load Balance Efficiency} = \frac{\text{Average}}{\text{Maximum}} \times 100\%$$

- The `imbalance_c1000000` test case has a small number of circles with imbalance size distribution. As the number of npernode increases and the target weight required by each process decreases, load imbalance is more likely to occur, as shown in Figure 8.
- Other test cases have a larger number of circles with more uniform size distribution, resulting in higher overall load balance efficiency, but there is a tendency for it to decrease as the number of npernode increases. The `large_c4000000` and `large_c2000000` test cases show that the more amount of data and the more uniform the distribution, the less likely the load balance efficiency will decrease due to an increase in npernode.

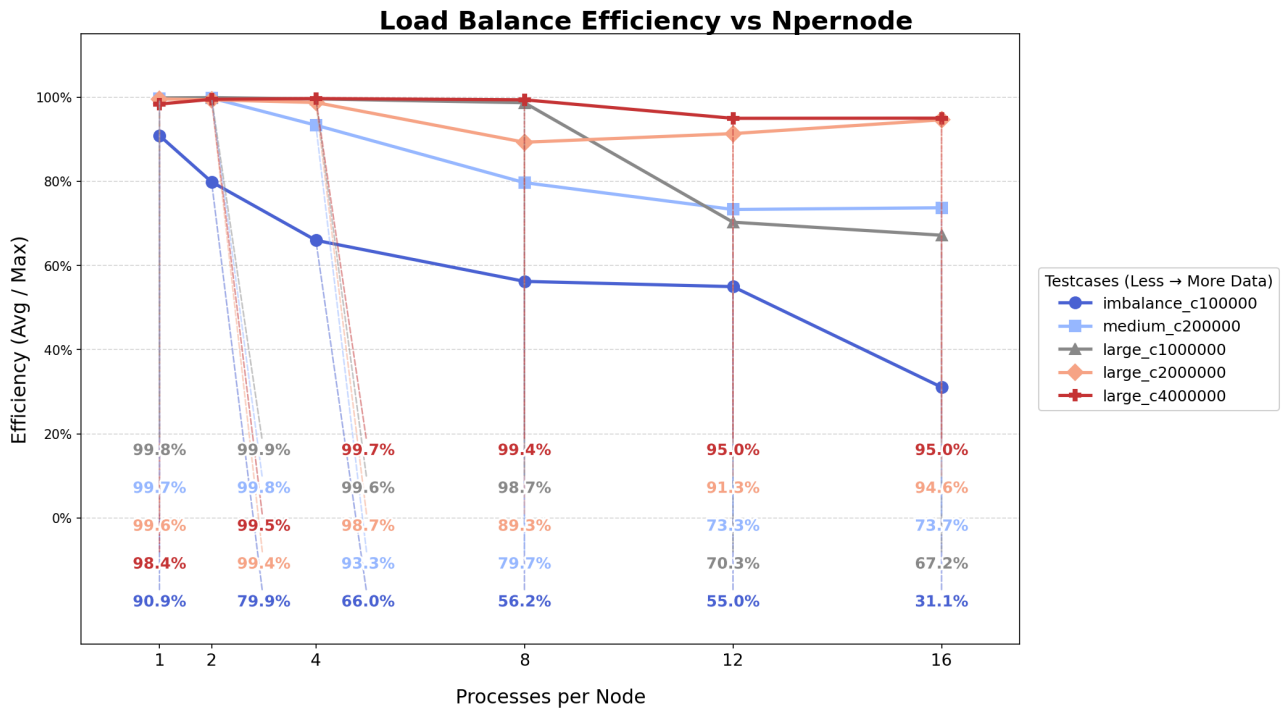


Figure 8: npernode vs. Load Balance Efficiency